

LoRA

LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

1. Introduction

기존 자연어처리에서는 보통 다음과 같은 방식으로 모델을 사용함.

대규모 데이터로 사전학습 → 특정 작업 데이터로 파인튜닝

예를 들어 GPT 같은 모델이 일반적인 언어 능력을 이미 학습했다고 하더라도 이를 뉴스 요약이나 SQL 생성 같은 특정 작업에 맞게 사용하려면 해당 작업의 데이터로 추가 학습하는 과정이 필요함.

사전학습된 모델의 파라미터를 Φ_0 라고 하면 파인튜닝 이후에는 $\Phi_0 + \Delta\Phi$ 가 됨.

- Φ_0 : 사전학습된 원래 모델의 파라미터
- $\Delta\Phi$: 특정 작업을 학습하면서 새롭게 생긴 변화량

기존 **Full Fine-Tuning**은 모델 전체 파라미터를 업데이트함.

이 방식은 모델이 작을 때는 큰 문제가 아니었지만 GPT-3처럼 엄청 많은 파라미터를 가진 모델에서는 문제가 커짐.

예를 들어 작업마다 전체 모델을 파인튜닝하면

- 요약용 모델
- SQL 생성용 모델
- 질의응답용 모델
- 번역용 모델을 사실상 각각 따로 저장해야 함.

따라서 모델이 커질수록

- 학습에 필요한 GPU 메모리 증가
- 작업별 모델 저장 공간 증가
- 여러 작업 사이의 모델 전환 비용 증가
- 배포 및 운영 비용 증가 문제가 발생함.

기존에도 이를 해결하려는 방법은 있었음.

대표적으로

- 일부 파라미터만 학습하는 방법
- Adapter를 추가하는 방법
- Prefix / Prompt를 학습하는 방법 등이 존재함.

하지만 Adapter는 기존 Transformer 사이에 새로운 연산을 추가하기 때문에 **추론 속도가 느려질 수 있고**, Prefix 계열은 학습 가능한 특수 토큰이 입력 길이를 차지한다는 문제가 있음.

LoRA : 특정 작업을 학습하면서 발생하는 가중치 변화량 ΔW 가 실제로는 모든 방향으로 자유롭게 변화할 필요가 없고 매우 적은 수의 중요한 방향만 바뀌면 충분할 수 있음.

= 원래 모델 자체를 **Low-Rank**로 만드는 것이 아니라 파인튜닝 과정에서 필요한 변화량을 **Low-Rank**로 표현하는 것

Rank

행렬이 가지고 있는 독립적인 정보 방향의 수

예를 들어 $\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$ 에서 두 번째 행은 첫 번째 행의 2배니까 실제로 독립적인 정보 방향은 하나뿐 → rank = 1

LoRA는 거대한 가중치 변화 행렬이라고 하더라도 실제로 필요한 변화가 몇 개의 중요한 방향에만 집중되어 있다면 거대한 행렬 전체를 직접 학습할 필요가 없다고 봄.

예를 들어 $W \in \mathbb{R}^{1000 \times 1000}$ 면 전체 행렬에는 $1000 \times 1000 = 1,000,000$ 개의 값이 존재하지만

변화량을 $\Delta W = BA$ 로 표현하고 $B \in \mathbb{R}^{1000 \times 4}$, $A \in \mathbb{R}^{4 \times 1000}$ 로 만들면 학습해야 하는 파라미터는 $1000 \times 4 + 4 \times 1000 = 8000$ 개뿐임.

그런데 두 행렬을 곱하면 $BA \in \mathbb{R}^{1000 \times 1000}$ 이 되므로 원래 변화량과 동일한 크기의 행렬을 만들 수 있음.

** 이 변화량은 **최대 rank 4의 제한된 구조**를 가지게 됨.

2. Problem Statement

사전학습된 언어 모델: $P_{\Phi}(y|x)$

- x : 모델 입력
- y : 모델이 생성해야 하는 출력
- Φ : 모델 전체 파라미터

ex) 자연어를 SQL로 변환하는 작업 $\rightarrow x$ =자연어 질문, y =SQL 문장

학습 데이터: $Z = \{(x_i, y_i)\}_{i=1, \dots, N}$ 형태의 입력-정답 쌍

Full Fine-Tuning

기존 Full Fine-Tuning의 목적함수:

$$\max_{\Phi} \sum_{(x,y) \in Z} \sum_{t=1}^{|y|} \log P_{\Phi}(y_t | x, y_{<t})$$

= 모든 학습 데이터의 모든 정답 토큰에 대해 올바른 토큰이 나올 확률을 높이도록 전체 모델 파라미터 Φ 학습

- y_t : 정답 문장의 t 번째 토큰
- $y_{<t}$: t 번째 이전까지 나온 정답 토큰
- $P_{\Phi}(y_t | x, y_{<t})$: 입력과 이전 토큰이 주어졌을 때 정답 토큰 y_t 가 나올 확률

ex) I love pizza 에서 pizza 를 예측 $\rightarrow y_t = \text{pizza}, y_{<t} = \text{I love}$

Full Fine-Tuning: $\Phi_0 \rightarrow \Phi_0 + \Delta\Phi$, 이때 $|\Delta\Phi| = |\Phi_0|$

즉 원래 모델이 175B 파라미터라면 작업별 업데이트도 사실상 175B 규모

Parameter-Efficient Fine-Tuning

Parameter-Efficient Fine-Tuning에서는 전체 변화량 $\Delta\Phi$ 를 직접 학습하지 않고 훨씬 작은 파라미터 집합 Θ 를 이용함.

$\Delta\Phi = \Delta\Phi(\Theta)$ 이고 $|\Theta| \ll |\Phi_0|$ 가 되도록 하는 것이 목표임.

이에 따라 목적함수는 $\max_{\Theta} \sum_{(x,y) \in Z} \sum_{t=1}^{|y|} \log P_{\Phi_0 + \Delta\Phi(\Theta)}(y_t | x, y_{<t})$ 가 됨.

위의 식과 거의 같지만 중요한 차이는 $\max_{\Phi}$ 가 아니라 $\max_{\Theta}$ 라는 점임.

즉 모델 전체를 학습하는 것이 아니라 작은 추가 파라미터만 학습함

Existing Solutions

기존 Parameter-Efficient 방법에는 크게 Adapter 계열과 Prefix 계열이 있었음.

1. Adapter

Transformer의 기존 layer 사이에 작은 신경망을 추가하는 방식

기존 Layer → Adapter → 기존 Layer

학습해야 할 파라미터는 적지만 Adapter 연산이 기존 모델 계산에 순차적으로 추가됨.

따라서 추론할 때도 Adapter를 계속 계산해야 함.

특히 온라인 서비스처럼 batch size가 작으면 GPU 병렬화를 충분히 활용하기 어렵기 때문에 지연이 커질 수 있음.

논문의 실험에서는 GPT-2 Medium에서 조건에 따라 약 20~30% 이상의 추론 지연이 발생함.

2. Prefix Tuning

모델 입력 앞에 학습 가능한 특수 토큰 추가

[p1,p2,p3,...]+실제 입력처럼 사용 → 모델 자체는 대부분 고정하고 p_i 와 같은 표현만 학습

하지만

- 최적화가 어려울 수 있음
- 특수 토큰이 입력 sequence 길이를 차지함
- 학습 가능한 파라미터를 많이 늘린다고 성능이 계속 향상되지 않음

LoRA는 입력에 토큰을 추가하지 않고 **모델 내부 가중치의 변화량을 학습하는 방식**이기 때문에 입력 sequence 길이를 소비하지 않음.

LoRA

$$W = W_0 + \Delta W$$

기존 Full Fine-Tuning에서는 ΔW 전체를 직접 학습했는데

LoRA에서는 $\Delta W = BA$ 로 표현함.

이때 $W_0 \in \mathbb{R}^{d \times k}$ 이고 $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$ 에서

행렬곱의 크기를 보면

$$(d \times r)(r \times k) = d \times k \text{ 이므로}$$

$BA \in \mathbb{R}^{d \times k}$ 가 되어 원래 가중치 W_0 와 더할 수 있음.

1. Low-Rank Update

원래 Dense Layer의 forward는 $h = W_0x$ 인데

LoRA를 적용하면 $h = W_0x + \Delta Wx$ 이고

$\Delta W = BA$ 이므로

$h = W_0x + BAx$ 가 됨. = 기존 모델의 출력+LoRA가 만든 보정값

LoRA는 새로운 모델을 처음부터 만드는 것이 아니라 **이미 학습되어 있는 모델에 특정 작업에 필요한 작은 보정값을 추가하는 방식**

Full Fine-Tuning에서는 $\Delta W \in \mathbb{R}^{d \times k}$ 이므로 학습해야 하는 파라미터 수가 dk 임

반면 LoRA에서는 **A**: $r \times k$, **B**: $d \times r$ 이므로 $rk + dr$ 개만 학습하면 됨.

만약 $d=k$ 면 Full Fine-Tuning은 d^2 개를 학습하지만 LoRA는 $2dr$ 개만 학습함.

r 이 d 보다 매우 작기 때문에 학습 파라미터가 크게 감소!

2. Initialization

$A \sim \mathcal{N}(0, \sigma^2)$ = A는 Gaussian 분포에서 랜덤하게 초기화, $B=0$ 으로 초기화함.

따라서 학습 시작 시점에는 $BA=0$ 이고 $h = W_0x$ 가 됨.

즉 **LoRA를 붙인 순간에는 기존 사전학습 모델과 완전히 동일하게 동작하고** 학습이 진행되면서 필요한 변화량만 점차 추가됨.

3. Scaling

실제 LoRA에서는 BAx 를 그대로 사용하는 것이 아니라 $\frac{\alpha}{r}BAx$ 로 scaling함.

따라서 실제 forward는 $h = W_0x + \frac{\alpha}{r}BAx$ 가 됨.

- r : LoRA rank
- α : scaling constant

r 을 변경할 때 LoRA 업데이트의 크기가 크게 달라지는 것을 줄이기 위해 사용

Full Fine-Tuning과의 관계

LoRA의 rank를 계속 크게 만들면 $r \rightarrow \min(d, k)$ 가 되고 BA 가 표현할 수 있는 변화 공간도 점점 커짐.

따라서 충분히 큰 r 에서는 일반적인 Full Fine-Tuning의 ΔW 와 유사한 표현력을 가지게 됨.

즉 LoRA는 Full Fine-Tuning과 완전히 별개의 방법이라기보다 **Full Fine-Tuning**이 허용하는 가중치 변화 공간을 **low-rank** 구조로 제한한 방법

Inference

학습 중에는 $W_0x + \frac{\alpha}{r}BAx$ 를 계산하지만 학습이 끝나면 $W' = W_0 + \frac{\alpha}{r}BA$ 로 미리 합칠 수 있음.

그 이후에는 추론할 때 $h=W'x$ 만 계산하면 됨.

따라서 Adapter처럼 추가 layer를 계산할 필요가 없고 추론 단계에서 추가 **latency**가 발생하지 않음.

작업을 바꿀 때도 전체 모델을 교체할 필요가 없음.

요약용 LoRA가 B_1A_1 이라면 $W = W_0 + B_1A_1$ 이고, SQL용 LoRA로 변경하려면 기존 LoRA를 제거하고 $W' = W_0 + B_2A_2$ 를 적용하면 됨.

= 하나의 거대한 기본 모델을 공유하고 작업마다 작은 LoRA만 따로 저장할 수 있음.

4. LoRA in Transformer

Transformer의 Self-Attention에는 대표적으로 네 종류의 가중치가 존재함.

W_q, W_k, W_v, W_o

- W_q : Query 생성
- W_k : Key 생성
- W_v : Value 생성
- W_o : Attention 결과를 출력 공간으로 변환

Self-Attention에서는 $Q = XW_q, K = XW_k, V = XW_v$ 를 만든 뒤

$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$ 를 계산함.

LoRA는 이 가중치 중 원하는 일부에 적용할 수 있음.

예를 들어 W_q, W_v 에 적용하면 $W_q = W_{q0} + B_qA_q, W_v = W_{v0} + B_vA_v$ 가 됨.

논문에서는 단순성과 파라미터 효율성을 위해 **Attention 가중치에만 LoRA를 적용하고 MLP는 고정함**.

주요 실험에서는 대부분 W_q, W_v 에 LoRA를 적용함.

Transformer의 가중치가 $d_{\text{model}} \times d_{\text{model}}$ 이라면 LoRA 하나의 학습 가능한 파라미터 수는 $2d_{\text{model}}r$ 임.

(A와 B 두 행렬이 있기 때문에 2가 붙음.)

전체 LoRA 파라미터 수는 $|\Theta| = 2 \times \hat{L}_{\text{LoRA}} \times d_{\text{model}} \times r$ 로 표현되는데

여기서 $\hat{L}_{\text{LoRA}}$ 는 LoRA를 적용한 weight matrix의 개수

Limitation

LoRA를 기존 가중치에 W0+BA 형태로 merge해 놓은 경우 하나의 batch 안에서 샘플마다 서로 다른 LoRA를 적용하기 어려움.

예를 들어 한 batch에서

- 샘플 1 → 요약 LoRA
- 샘플 2 → 번역 LoRA
- 샘플 3 → SQL LoRA 처럼 서로 다른 작업을 동시에 처리하려는 경우임.

LoRA를 merge하지 않고 별도 계산 경로로 유지하면 가능하지만 이 경우에는 추가 연산 비용을 감수해야 함.

5. Experiments

실험에서는 LoRA가 특정 모델이나 특정 작업에만 효과적인지 확인하기 위해 여러 모델과 데이터셋을 사용함.

결과: LoRA가 Full Fine-Tuning보다 훨씬 적은 수의 파라미터를 학습하면서도 비슷하거나 더 좋은 성능을 기록함

- GPT-3 175B에서 Full Fine-Tuning은 약 175B 규모의 파라미터를 학습하지만 일부 LoRA 설정에서는 약 **4.7M개의 파라미터만 학습함.**

그럼에도 WikiSQL, MNLI, SAMSum에서 Full Fine-Tuning과 비슷하거나 더 좋은 성능을 기록함.

즉 모델을 새로운 작업에 적응시키기 위해 반드시 전체 파라미터를 변경할 필요는 없다는 결과

- Adapter와 비교했을 때는 Adapter가 모델에 새로운 연산 경로를 추가하기 때문에 작은 batch size에서 추론 지연이 발생함.
- LoRA는 학습 후 가중치를 merge할 수 있기 때문에 추가 latency가 없음.

- Prefix 계열에서는 학습 가능한 특수 토큰 수를 늘린다고 성능이 계속 좋아지지 않았으며, 오히려 일정 수준 이후 성능이 떨어지는 경우가 나타남.

반면 LoRA는 rank와 학습 파라미터를 늘려도 성능이 비교적 안정적으로 유지됨.

6. Related Works

LoRA와 관련된 기존 연구를 크게 네 가지 흐름으로 설명함.

Transformer Language Models

BERT, GPT-2, GPT-3처럼 대규모 데이터로 Transformer를 사전학습한 뒤 특정 downstream task에 적응시키는 방식

Prompt Engineering and Fine-Tuning

GPT-3처럼 큰 모델은 몇 개의 예시만 prompt에 넣어도 어느 정도 새로운 작업을 수행할 수 있지만, 성능이 prompt 구성에 크게 영향을 받음.

또 논문의 Appendix 실험에서는 GPT-3에서도 Few-Shot보다 Fine-Tuning의 성능이 크게 높았음.

따라서 큰 모델이라도 여전히 파라미터 업데이트가 필요할 수 있음.

Parameter-Efficient Adaptation

Adapter나 Prefix Tuning처럼 전체 모델을 학습하지 않고 일부 파라미터만 학습하는 방법임.

LoRA 역시 이 범주에 포함되지만, Adapter와 달리 학습한 가중치를 기존 weight에 merge할 수 있어 추가 inference latency가 없다는 차이가 있음.

Low-Rank Structures

기존 머신러닝에서도 Low-Rank 구조를 이용한 연구는 많이 존재했는데, 주로 모델 자체의 가중치를 Low-Rank로 만들거나 압축하는 방식이었음.

LoRA의 차이점은 W 자체를 Low-Rank로 만드는 것이 아니라 ΔW 을 사용하여 **사전학습된 모델을 특정 작업에 적응시키면서 발생하는 변화량만 Low-Rank로 제한함**

7. Understanding Low-Rank Updates

왜 작은 rank만으로도 충분한지 분석할까?

- Transformer에서 어떤 가중치에 LoRA를 적용하는 것이 좋은가?
- ΔW \Delta W는 실제로 매우 낮은 rank만 필요로 하는가?
- LoRA가 학습한 ΔW 와 기존 가중치 W 는 어떤 관계가 있는가?

Weight Selection

동일한 학습 파라미터 예산을 두고 W_q, W_k, W_v, W_o 및 여러 조합을 비교함.

예를 들어 하나의 weight에 LoRA를 적용하면 높은 rank를 사용할 수 있지만 여러 weight에 적용하면 같은 파라미터 예산에서 각 rank는 낮아져야 함.

실험 결과 하나의 weight에 높은 rank를 집중하는 것보다 W_q, W_v 등 여러 weight에 작은 rank를 나누어 적용하는 것이 전반적으로 좋은 결과를 보임. (특히 W_q, W_v 조합이 효과적)

즉 LoRA에서는 단순히 rank를 크게 만드는 것보다 **어느 weight에 LoRA를 적용하느냐도 중요함.**

Rank r

$r=1,2,4,8,64$ 등을 비교

일반적으로는 rank가 높아질수록 표현력이 증가하므로 성능도 계속 향상될 것이라고 생각할 수 있는데 실제 실험에서는 그렇지 않았음.

GPT-3에서 W_q, W_v 를 함께 학습한 경우 $r=1$ 만으로도 높은 성능을 기록함.

이는 ΔW 가 실제로 필요로 하는 핵심 정보 방향의 수가 매우 적을 수 있음을 의미하고, 논문에서는 이를 **낮은 intrinsic rank**라고 해석

**** 모든 작업에서 $r=1$ 이면 충분하다는 의미는 아님.**

예를 들어 사전학습에 사용되지 않은 완전히 새로운 언어를 학습하는 등 모델 자체가 크게 변해야 하는 작업에서는 더 큰 rank가 필요할 수 있음.

따라서 핵심은 **LoRA의 최적 rank가 항상 1이라는 것이 아니라, 필요한 rank가 원래 모델의 차원에 비해 매우 작을 가능성이 높다는 것**

SVD

low-rank 구조가 존재하는지 확인하기 위해 **SVD(Singular Value Decomposition)**를 사용

어떤 행렬 A 는 $A = U\Sigma V^T$ 로 분해할 수 있음.

- U, V : 행렬이 가지고 있는 주요 방향

- Σ : 각 방향이 얼마나 중요한지를 나타내는 singular value (클수록 해당 행렬에서 중요한 방향)

질문: rank 8 LoRA와 rank 64 LoRA가 결국 비슷한 핵심 방향을 학습하는가?

저자들은 같은 task에서 학습한 $A_{r=8}$ 과 $A_{r=64}$ 를 비교함.

Subspace Similarity

두 LoRA가 학습한 방향이 얼마나 겹치는지 측정: $\phi(A_{r=8}, A_{r=64}, i, j) = \frac{\|U_{A_{r=8}}^{iT} U_{A_{r=64}}^j\|_F^2}{\min(i, j)} \in [0, 1]$

- U^i : SVD에서 얻은 방향 중 상위 i 개의 중요한 방향을 의미함.
- $\|\cdot\|_F$: Frobenius Norm

행렬 M 에 대해 $\|M\|_F = \sqrt{\sum_{i,j} M_{ij}^2}$ 로 계산 = 행렬 전체의 크기를 하나의 값으로 나타내는 방법

Subspace Similarity에서 $\phi=1$ 이면 두 subspace가 완전히 겹치고 $\phi=0$ 이면 두 공간이 완전히 분리되어 있다는 의미

실험 결과 $r=8$ 과 $r=64$ 에서 가장 중요한 상위 singular direction이 상당히 겹침.

즉 rank를 64까지 크게 주더라도 실제로 중요한 정보는 몇 개의 핵심 방향에 집중되어 있고 나머지 방향은 상대적으로 학습 과정에서 발생한 noise일 가능성이 있음.

이 결과는 왜 작은 rank로도 높은 성능을 낼 수 있는지를 설명함.

서로 다른 random seed로 학습한 두 $r=64$ LoRA도 비교했는데 초기값이 달라도 일부 중요한 singular direction이 공통적으로 나타났지만 서로 다른 random Gaussian 행렬끼리는 이러한 공통 방향이 거의 나타나지 않음.

따라서 LoRA가 찾은 핵심 방향이 단순히 무작위로 생긴 것이 아니라 downstream task와 관련된 의미 있는 방향일 가능성을 보여줌.

ΔW & W

질문: LoRA가 기존 모델에 완전히 새로운 정보를 추가하는 것인지, 아니면 기존 모델에 이미 존재하는 정보를 강조하는 것인지?

LoRA의 변화량 ΔW 와 기존 가중치 W 의 관계를 분석함.

ΔW 를 SVD하면 $\Delta W = U\Sigma V^T$ 가 됨.

저자들은 기존 WW 를 ΔW 가 사용하는 방향에 투영하여 $U^T W W$ 를 계산

(= LoRA가 중요하게 사용하는 방향이 기존 모델에도 원래 어느 정도 존재하고 있었는가?를 확인)

분석 결과

1. ΔW 는 random matrix보다 기존 WW 와 더 큰 관계를 가짐. = LoRA가 완전히 무관한 새로운 방향을 만드는 것은 아님.
2. 그렇다고 기존 WW 에서 이미 가장 강하게 나타나는 방향을 그대로 반복하지도 않음.
3. 기존 모델에 존재하기는 하지만 상대적으로 약하게 표현되었던 방향을 LoRA가 크게 강화하는 경향이 나타남.

이를 통해 저자들은 LoRA가 사전학습 과정에서 이미 학습되었지만 특정 downstream task에서는 충분히 강조되지 않았던 feature를 선택적으로 증폭할 가능성이 있다고 해석함.

Amplification Factor

논문의 $r=4$ 분석에서는 $\|\Delta W_q\|_F = 6.91$ 이고 LoRA가 선택한 방향에서 기존 Wq 의 크기는 약 0.32였음.

따라서 $\frac{6.91}{0.32} \approx 21.5$ 가 됨.

즉 기존 모델에서 비교적 약하게 존재하던 특정 방향을 LoRA가 약 20배 정도 크게 강화한 셈

$$\text{Amplification Factor} = \frac{\|\Delta W\|_F}{\|U^T W W\|_F}$$

- 분모: LoRA가 선택한 방향이 기존 모델에서 원래 얼마나 강했는지
- 분자: LoRA가 그 방향에 얼마나 큰 변화를 추가했는지

$r=4$ 에서는 amplification factor가 약 20 수준이었지만 $r=64$ 에서는 약 2 수준으로 낮아짐.

저자들은 이것 역시 필요한 intrinsic rank가 낮다는 증거로 해석함.

작은 rank에서는 소수의 중요한 task-specific 방향에 변화가 집중되지만, rank가 너무 커지면 상대적으로 중요하지 않은 방향까지 함께 포함되기 때문임.

8. Conclusion

LoRA는 대규모 언어 모델을 특정 작업에 적응시키기 위해 전체 모델을 다시 학습하는 대신 $W = W_0 + \Delta W$ 에서 $\rightarrow$ 변화량을 $\Delta W = BA$ 로 표현함.

따라서 실제 forward는 $h = W_0x + \frac{\alpha}{r}BAx$ 가 됨.

- W_0 : 사전학습된 기존 가중치, 고정
- A, B : 학습하는 작은 Low-Rank 행렬
- r : LoRA rank
- α : LoRA 업데이트 크기를 조절하는 scaling 값

학습이 끝나면 $W' = W_0 + \frac{\alpha}{r}BA$ 로 기존 가중치에 합칠 수 있기 때문에 추론 과정에서 추가적인 계산 단계가 필요 X

LoRA의 핵심 장점

- 모델 전체가 아니라 작은 A, B 만 학습하기 때문에 학습 파라미터 수가 크게 감소함
- 대부분의 기존 파라미터에서 gradient와 optimizer state를 관리할 필요가 없어 GPU 메모리를 절약함
- 작업별로 전체 모델을 저장할 필요 없이 작은 LoRA checkpoint만 따로 저장하면 됨
- LoRA를 기존 weight에 merge할 수 있어 Adapter와 달리 추가 inference latency가 없음
- Prefix 계열처럼 특수 토큰을 넣지 않으므로 입력 sequence 길이를 소비하지 않음
- 실험에서 Full Fine-Tuning과 비슷하거나 더 높은 성능을 기록함
- Prefix Tuning 등 다른 Parameter-Efficient 방법과 결합할 수도 있음